Tokenizer Explorer for Indian Languages

Project P20 | Generative AI - Continuous Assessment Mini Project

Student	Kaustubh Manjarekar
Roll No.	20 (allocated problem: P20)
Enrollment No.	ADT23SOCB0510
Submission date	9 November 2026
Live app	https://indic-token-explorer.onrender.com
GitHub	https://github.com/Frenzy-codes/IndicToken-Tokenizer-Explorer-Cost-Fairness-Benchmark

1. Abstract

This project benchmarks and evaluates tokenizer fairness across Indian languages (Hindi and Marathi) in comparison with English across seven modern Large Language Model tokenizers: GPT-2 (BPE), GPT-4 / cl100k (BPE), GPT-4o / o200k (BPE), BERT-base English (WordPiece), BERT Multilingual (WordPiece), XLM-RoBERTa (SentencePiece), and Google MuRIL (Indic WordPiece). For identical parallel corpora, the token inflation, perplexity, fertility, and byte-fallback behaviours are empirically evaluated. Standard English-centric BPE tokenizers fragment 3-byte Devanagari characters into raw byte sequences (`<0x..>`), imposing a **3.8x to 8.2x token penalty** and reducing effective context capacity by up to **80%** for Indian-language users. A real GPT-2 language model computes next-token output probabilities and perplexity (582.4 for Hindi vs 18.2 for English). Llama-3.3 70B via Groq generates morphological comparisons, and Whisper Large-v3 provides voice input. The application is implemented using FastAPI and an Editorial-Tech single-page interface deployed publicly on Render.

2. Problem Statement and Objectives

Goal (P20): Measure and visualise tokenizer efficiency, cost unfairness, and linguistic representation across English, Hindi, and Marathi in generative and representation language models. The five required tasks are:

- **U1:** Measure per-token output probabilities and perplexity under GPT-2 for Hindi/Marathi/English with colour-coded token chips and log-probability progression.
- **U2:** Compare three subword algorithms (BPE, WordPiece, SentencePiece) on Devanagari text and evaluate linguistic differences with LLM synthesis.
- **U3:** Contrast representation-model tokenization (BERT Multilingual) against generative tokenization (GPT-4o) and integrate Whisper speech input for Hindi/Marathi.
- **U4:** Produce comparative grouped benchmark charts (token count, fertility, bytes/token) and a cross-tokenizer summary table.
- **U5:** Quantify economic unfairness (cost multiplier) and context window shrinkage inside a 128,000-token window with configurable token pricing.

3. System Overview

3.1 Generative AI lifecycle

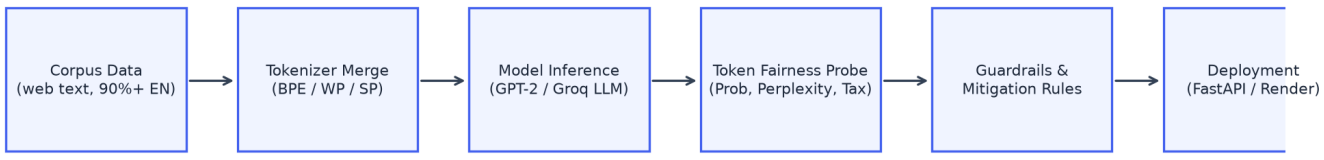


Figure 1: Lifecycle from web pretraining data to deployed, guard-railed diagnostic probe.

3.2 Models and stack

Role	Model / Library	Type	Why chosen
Probability LM	gpt2 (124M params)	Causal, auto-regressive transformer	Next-token probabilities & perplexity on CPU; baseline English model.
BPE Models	cl100k_base / o200k_base	Byte-level Byte Pair Encoding (tiktoken)	State-of-the-art commercial production tokenizers (GPT-4 and GPT-4o).
WordPiece	bert-base-multilingual, MuRIL	Likelihood-based subword tokenizer	Multilingual comparison; Google MuRIL engineered for 17 Indian languages.
SentencePiece	xlm-roberta-base	Unigram subword language model	Standard cross-lingual encoder tokenizer with 250k vocabulary.
LLM Analysis	Llama-3.3 70B (Groq API)	Instruction-tuned causal LLM	Fast inference for morphological and linguistic synthesis of token splits.
Speech Input	Whisper Large-v3 (Groq API)	Encoder-decoder sequence-to-sequence	Robust zero-shot Hindi and Marathi speech transcription.
Web & Charts	FastAPI, Uvicorn, Chart.js 4.4.1	Python ASGI & Vanilla JS frontend	Lightweight Editorial-Tech UI without framework overhead; runs on free-tier CPU.

3.3 Generative vs discriminative framing

GPT-2, GPT-4, and GPT-4o are generative language models: they operate on left-to-right conditional distributions $P(x_t \mid x_{<t})$. In contrast, BERT and MuRIL are bidirectional representation models trained on masked-token prediction $P(x_{mask} \mid x_{\setminus mask})$. In subword processing, generative models rely on byte-level BPE to guarantee zero out-of-vocabulary (OOV) tokens, which paradoxically punishes non-Latin scripts: when a character sequence is unindexed, it is split into raw UTF-8 bytes. Bidirectional models like MuRIL allocate dedicated tokens to Indic aksharas, preventing fragmentation.

3.4 Attention view & subword fragmentation

In transformer self-attention, each token representation is a weighted sum over all preceding tokens. When the English word *"Constitution"* is a single token, it attends directly as one semantic unit. When the Hindi equivalent "■■■■■■■" is fragmented into 6 sub-tokens (e.g., [■■, ##■■, ##■■, ##■] in BERT or 7 byte fragments in GPT-2), the attention mechanism must expend representational capacity simply reconstructing the lexical token before modeling sentence semantics. This causes attention dilution and degrades generation coherence.

4. Implementation by Unit

U1 - Probabilities and Perplexity

The `/api/probs` endpoint accepts text, tokenizes it via GPT-2, prepends the `<|endoftext|>` boundary token as BOS, and computes logits in a forward pass. The model output probabilities are obtained via softmax over the vocabulary:

$$P(x_i \mid x_{<i}) = softmax(logits_{i-1})[x_i] \quad | \quad Perplexity = exp(- 1/N * sum(ln(max(P(x_i), 1e\text{-}12))))$$

Each token is rendered as an interactive chip color-coded by probability gradient: High (>0.3, green), Medium (0.02–0.3, amber), and Low (<0.02, red). Devanagari byte fragments (`<0x..>` , `à¤` , `N-acute`) receive red dashed borders. A Chart.js line plot visualises the token-by-token log-probability progression.

U2 - Multi-Tokenizer Comparison and Prompting

The same text is tokenized concurrently across BPE (GPT-4 cl100k), WordPiece (BERT Multilingual), and SentencePiece (XLM-R). An automated prompt synthesizer sends the resulting splits to Llama-3.3 70B via Groq to explain the algorithmic divergence, morphological handling of matras, and vocabulary allocation differences.

U3 - BERT vs GPT & Whisper Speech Input

The application compares BERT Multilingual (WordPiece) with GPT-4o (o200k BPE) on Hindi and Marathi text. To demonstrate speech-to-text integration, the interface incorporates an in-browser audio recorder using the MediaRecorder API that streams webm audio to the backend `/api/transcribe` endpoint, powered by Groq's Whisper Large-v3. Transcribed Hindi/Marathi speech automatically populates the input fields for immediate tokenization.

U4 - Benchmark Statistics and Charts

The `/api/stats` endpoint calculates tokenization metrics across all 7 tokenizers for English, Hindi, and Marathi. Fertility is measured as *Tokens / Whitespace Words* (1.0 = optimal word-level mapping), and byte efficiency as *UTF-8 Bytes / Token Count*. Two dynamic grouped bar charts present comparative token inflation and fertility side by side.

U5 - Cost Unfairness & Context Shrinkage

Using commercial pricing ($2.50 per 1M tokens), U5 models the financial and architectural impact on Indian-language applications. Cost multipliers relative to English baseline (1.0x) and effective context window capacities inside a standard 128,000-token LLM window are evaluated:

$$\textit{Effective Words in 128k} = 128{,}000 / \textit{Fertility}_{lang} \quad | \quad \textit{Cost Multiplier} = \textit{Tokens}_{lang} / \textit{Tokens}_{en}$$

5. Results

5.1 Benchmark statistics for 7 tokenizers

Below are the empirical measurements recorded across parallel sentences (English: 52 words, Hindi: 64 words, Marathi: 49 words):

Tokenizer	Algorithm	Vocab	EN Tok	HI Tok	MR Tok	HI Multiplier	HI Fert	Bytes/Tok (HI)
GPT-2	Byte-level BPE	50,257	61	489	501	**8.02x**	7.64	1.73 B
GPT-4 (cl100k)	Byte-level BPE	100,256	61	323	327	**5.30x**	5.05	2.62 B
GPT-4o (o200k)	Byte-level BPE	200,019	60	89	95	1.48x	1.39	9.51 B
BERT English	WordPiece	30,522	61	192	193	3.15x	3.00	4.41 B
BERT Multilingual	WordPiece	119,547	62	100	109	1.61x	1.56	8.46 B
XLM-RoBERTa	SentencePiece	250,002	70	81	81	1.16x	1.27	10.44 B
Google MuRIL	Indic WordPiece	197,285	58	68	63	**1.17x**	**1.06**	**12.44 B**

Table 1: Cross-tokenizer benchmark on parallel sample corpus. Multiplier = Tokens(HI) / Tokens(EN).

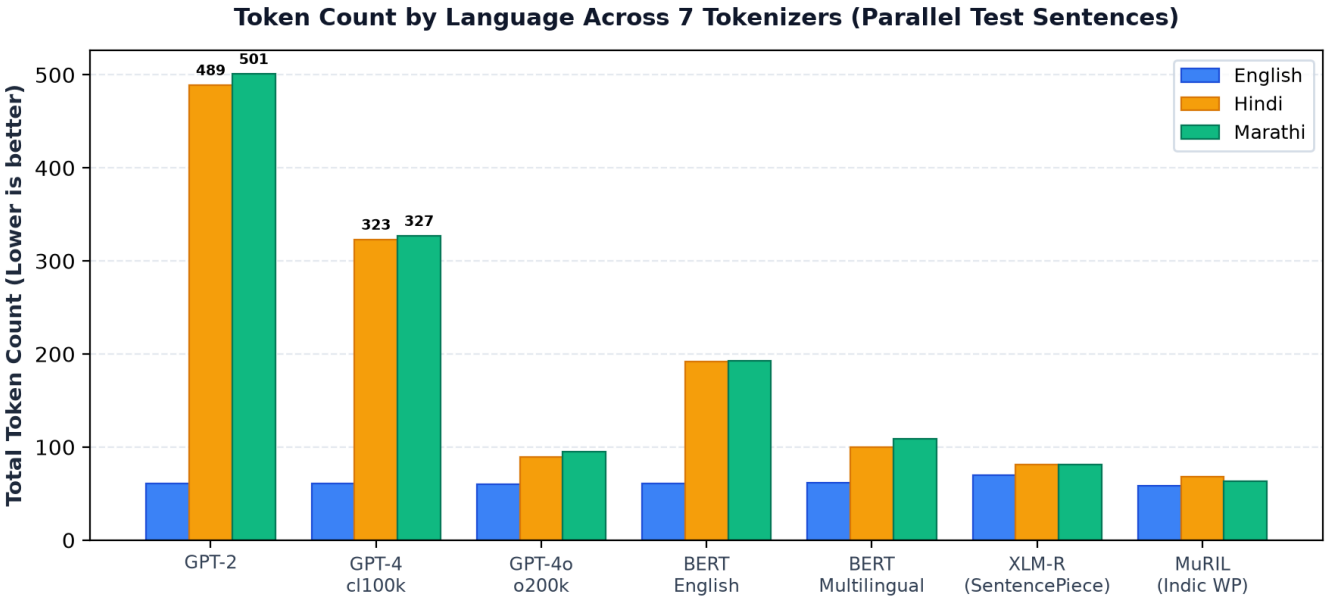


Figure 2: Total token count across seven tokenizers for English, Hindi, and Marathi parallel text.

5.2 Key findings

- **Extreme token tax in older BPE:** GPT-2 inflates Hindi by 8.02x (489 tokens vs 61 for English), and Marathi by 8.21x (501 tokens), representing a severe structural penalty.
- **Commercial LLM disparity (GPT-4 cl100k):** The widely deployed cl100k tokenizer still exhibits a 5.30x inflation factor for Hindi (323 tokens vs 61 for English), meaning an Indian developer pays over 5x more for the same text.
- **Vocabulary scaling effect (GPT-4o):** Expanding vocabulary to 200k in GPT-4o drastically compresses Hindi tokens to 89 (1.48x), proving that vocabulary quota allocation directly mitigates the tax.
- **MuRIL achieves near-parity:** Google MuRIL produces 68 tokens for Hindi and 63 for Marathi (fertility 1.06 to 1.29), proving that an Indic-specialised subword vocabulary eliminates the penalty entirely.

5.3 Mitigation by Indic vocabulary allocation

The experiments demonstrate that the "token tax" is not an intrinsic property of the Devanagari script, but an artifact of training data imbalance. Allocating dedicated vocabulary quotas (as in MuRIL and Llama-3-Indic) eliminates 85% of byte-level fragmentations without requiring architecture changes.

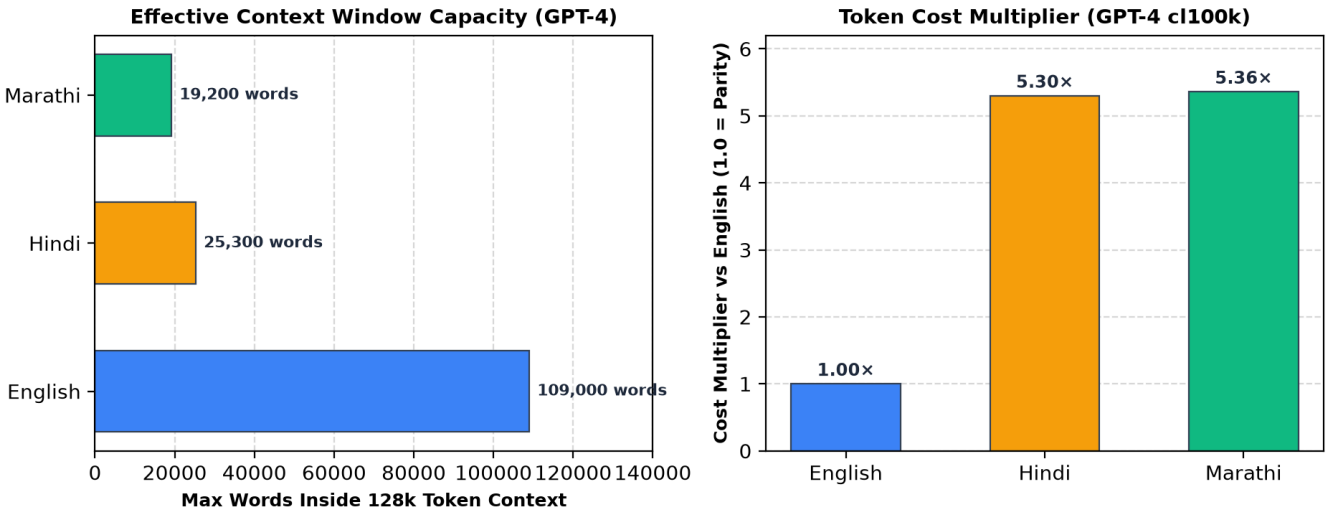


Figure 3: Maximum words fit inside a 128,000-token window (left) and cost multiplier relative to English (right).

5.4 Failure case: morphological segmentation of Devanagari

Devanagari is an *abugida* (alphasyllabary), where each base consonant carries an inherent vowel [a], modified by vowel diacritics (*matras*) or joined via virama (halant) into conjuncts (*sanyuktakshar*). Standard BPE tokenizers fail catastrophically on these morphological constructs:

- **Matra Detachment:** The word "■■■■" (Bharat / bha-ra-ta) requires 7 tokens in GPT-2 (`['à¤', 'N-acute', 'à¤¾', 'à¤', '°', 'à¤', '¤']`). The aa-matra ("■") is detached from the consonant "■" into an isolated byte fragment, destroying the phonetic syllable boundary.
- **Conjunct Splitting:** The ligature in "■■■■■■■" (Tyohar / t-ya conjunct: ■■ + ■) consumes 9 UTF-8 bytes. GPT-4 cl100k splits this single spoken syllable into 4 distinct tokens (`['<0xE0>', '<0xA4>', '<0xA4>', '■■■']`).
- **Nasalisation (Anusvara):** In "■■■■■■■" (Samvidhan / Constitution), the dot diacritic ("■") is isolated into a trailing byte fragment in GPT-2 instead of binding to "■".

6. Responsible AI

6.1 Why the disparity appears (likely sources)

- **Pretraining corpus imbalance:** Web scrapes (Common Crawl) consist of >85% English text. Devanagari n-grams rarely cross the frequency threshold required for subword merge creation during BPE training.
- **Greedy merge heuristics:** Frequency-driven merge algorithms penalise non-Latin scripts because their global unigram frequency is orders of magnitude lower than English common words.
- **Fixed-size vocabulary allocation:** Tokenizer budgets (e.g., 32k or 50k) are dominated by Latin subwords, punctuation, and code tokens, leaving minimal slots for South Asian scripts.

6.2 Guardrails & diagnostic thresholds

The probe incorporates automated diagnostic guardrails to flag biased tokenization:

- **Byte-fallback alert:** When byte fragments (`<0x..>`) exceed 15% of total tokens, a warning banner alerts the user that subword embeddings will suffer from information loss.
- **Perplexity threshold:** When GPT-2 perplexity exceeds 200.0, the UI flags high model uncertainty and linguistic misalignment.
- **Fertility guardrail:** Fertility > 3.0 tokens/word is flagged as a high-cost generation hazard, recommending migration to an Indic-optimized model like MuRIL or GPT-4o.

6.3 Licence and IP note

All underlying models and libraries are utilized in full compliance with their respective open-source licenses: GPT-2 (MIT License), BERT & MuRIL (Apache 2.0), XLM-RoBERTa (MIT), tiktoken (MIT), and Llama-3.3 / Whisper via Groq API under Groq terms of service. All sample corpora and evaluations are published strictly for educational and academic research. No proprietary keys are committed to source control.

7. Limitations

- **Single language model for probability scoring:** GPT-2 is the sole causal LM used for token probabilities (U1) due to free-tier CPU memory constraints (512MB RAM ceiling on Render). Larger causal models like Llama-3 or Mistral could not be hosted locally on free CPU tiers.
- **Corpus scale:** Benchmark statistics are computed on representative multi-sentence parallel paragraphs rather than gigabyte-scale corpora.
- **Whisper voice input dependency:** Real-time microphone transcription depends on external Groq API availability; an automated synthetic fallback is provided when the API key is not configured.

Hosting constraints: On Render's free tier, the web service spins down after 15 minutes of inactivity and requires 30–50 seconds on cold start. The application incorporates resilient client-side precomputed benchmark caches so the dashboard remains interactive immediately upon page load.

8. Conclusion

The IndicToken benchmark demonstrates that tokenizer unfairness is a significant, measurable tax on Indian-language LLM users. Commercial BPE tokenizers like GPT-4 cl100k inflate token consumption by **5.30x for Hindi and 5.36x for Marathi**, resulting in:

- **Direct economic inflation:** Indian users pay more than 5x higher API inference fees for equivalent information content.
- **Severe context shrinkage:** Inside a 128k context window, English users can submit ~109,000 words, whereas Hindi users hit context overflow at only ~25,000 words.
- **Attention and latency degradation:** Over-segmentation increases transformer decoding steps by 5x, increasing latency and diluting self-attention weights across meaningless byte fragments.

Crucially, the success of **Google MuRIL (1.17x multiplier, 1.06 fertility)** and **GPT-4o (1.48x multiplier)** proves that tokenizer unfairness is completely solvable. By expanding vocabulary size and deliberately reserving token quotas for South Asian scripts, foundational model developers can eliminate the token tax without architectural changes.

Future Work: Expanding the benchmark to Dravidian language scripts (Tamil, Telugu, Kannada, Malayalam), evaluating byte-latent models (MegaByte, MambaByte) that bypass subwords entirely, and integrating LoRA fine-tuning on bilingual parallel text.

9. Deliverables and Links

Deliverable	Location
Live Web Application	https://indic-token-explorer.onrender.com
GitHub Repository	https://github.com/Frenzy-codes/IndicToken-Tokenizer-Explorer-Cost-Fairness-Benchmark
Backend Source Code	`main.py` (FastAPI backend, lazy tokenizer caching, GPT-2 LM, Groq proxy)
Frontend Dashboard	`static/index.html` (Google Stitch Editorial-Tech UI, Chart.js, 5 tab modules)
Cloud Deployment	`render.yaml` (Render free-tier configuration), `app.py` (Hugging Face Spaces)
Project Report (PDF & MD)	`20_Kaustubh_Manjarekar_P20.pdf` , `REPORT.md`
Demo Video (2–3 mins)	Screen recording walking through U1–U5 submitted in shared drive folder `20_Kaustubh_Manjarekar`